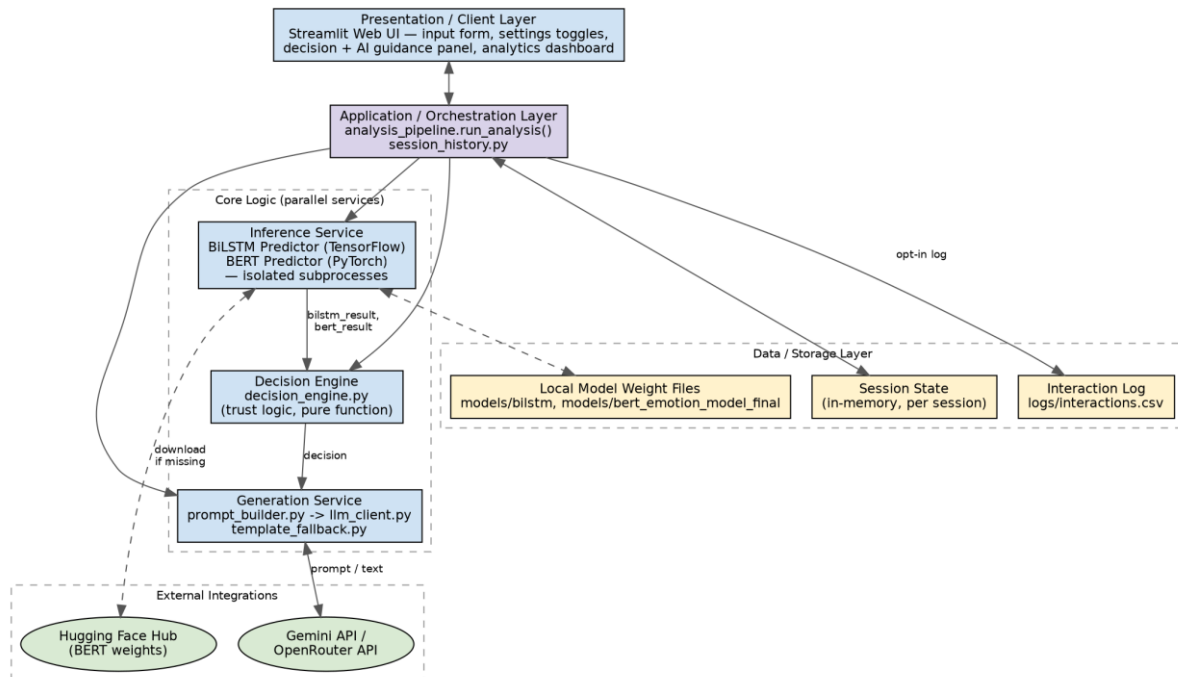


Solution Architecture

Date	14 July 2026
Team ID	SWTID-2026-7049
Project Name	Emotion Detection & Learning Support Engine (AI Learning Assistant)
Maximum Marks	5 Marks

Solution Architecture Diagram

Maps the actual layered structure of the app: presentation, orchestration, the two core services that run in parallel (inference and, downstream of it, decision + generation), external integrations, and the data/storage layer.



Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Renders the input form, settings toggles, decision/AI-guidance panel, model comparison, and analytics dashboard; owns Streamlit session state.	Streamlit 1.37.1, Plotly 5.24.1
Orchestration Layer	Single entry point (run_analysis) that sequences prediction -> decision -> generation and hands the combined result back to the UI; also drives session-history recording.	Python — analysis_pipeline.py, session_history.py
Inference Service	Loads and runs the two independently-trained emotion classifiers; each framework runs in its own isolated subprocess to avoid a confirmed TensorFlow/PyTorch segfault when loaded together.	TensorFlow (tensorflow-cpu), PyTorch + Transformers, cached via @st.cache_resource
Decision Engine	Pure function that combines both models' outputs into a final emotion, trust level, and reason code, calibrated against a 770-row validation analysis.	Python — decision_engine.py, no framework dependency

Component Name	Description / Role in Architecture	Technologies Used
Generation Service	Builds an emotion- and field-aware prompt and calls the configured LLM provider; falls back to a deterministic template if AI is off or the call fails.	google-generativeai (Gemini) / openai SDK via OpenRouter, prompt_builder.py, template_fallback.py
External Integrations	Supplies the language generation and, when local weights aren't present, the BERT model weights.	Gemini API, OpenRouter API, Hugging Face Hub
Data / Storage Layer	Holds per-session state in memory, local/downloaded model weight files, and the opt-in interaction log.	Streamlit session_state, local filesystem, CSV (logs/interactions.csv)